

Assignment 3 | Bridge Pattern

Student: Abylai Duisim

Group: SE-2530

Topic: B (Notifications)

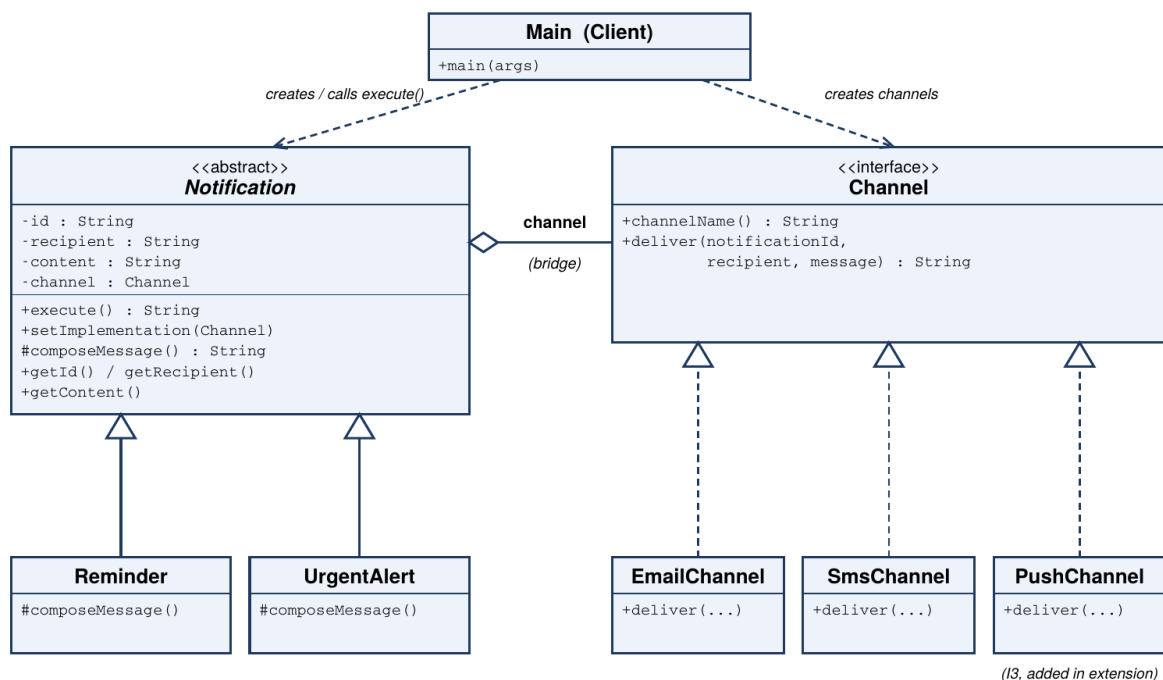
Repository: <https://github.com/Abylaisdb/Assignment-3-Bridge-Pattern>

Base commit (working Email/SMS version): c2440f6ac178888266e791fb63d840ecd7d0938f

1. Two dimensions and why Bridge fits

The application varies along two independent dimensions. The first is **what kind of notification** is created: a normal *Reminder* or an *UrgentAlert* marked URGENT. The second is **how it is delivered**: an email envelope, a single-line SMS or a push notification. A plain inheritance solution would need one class per pair (*EmailReminder*, *SmsReminder*, *EmailUrgentAlert*, ...), which grows multiplicatively with every new kind or channel. With Bridge, *Notification* (abstraction) owns a *Channel* (implementor) reference and delegates delivery to it. Each hierarchy grows on its own, any notification can be used with any channel, and the channel can even be replaced on a living object at runtime.

2. UML class diagram



3. Annotated code excerpts

(a) Separation of responsibilities. *Notification* decides *what* is sent, *Channel* decides *how* it looks when delivered. Changing an email format never touches notification logic.

```
// notification/Notification.java
public String execute() {
    return channel.deliver(id, recipient, composeMessage());
}
```

(b) Meaningful names. The names state intent: `composeMessage` builds text, `deliver` sends it, `setImplementation` swaps the bridge, so the code reads almost like the requirements.

```
// notification/UrgentAlert.java
@Override
protected String composeMessage() {
    return "URGENT: " + getContent();
}
```

(c) Small focused methods. Each helper in `Main` does one thing, so the check logic is easy to read and to change.

```
// Main.java
private static boolean record(boolean ok) {
    total++;
    if (ok) { passed++; }
    return ok;
}

private static String verdict(boolean ok) { return ok ? "PASS" : "FAIL"; }
```

(d) Avoidance of duplication. The delivery workflow is written once in `execute()` and only the message differs in subclasses. In the demo one method serves T1-T4, T6 and T7, instead of six copies.

```
// Main.java
checkCombination("T1", new EmailChannel(), Main::reminder, "EMAIL | id=N-001 | ...");
checkCombination("T6", new PushChannel(), Main::reminder, "PUSH | id=N-001 | ...");
```

(e) Encapsulation of state. Domain data is private and final, so it cannot change after construction. The only mutable part is the bridge, and it changes only through `setImplementation`, which is why T5 can prove that data stays the same.

```
// notification/Notification.java
private final String id;
private final String recipient;
private final String content;
private Channel channel; // the bridge, replaceable only via setImplementation(...)
```

4. Extension, trade-off and comparison with Adapter

How I3 was added. After committing the working Email/SMS version (base commit above), I created `PushChannel` as one new class implementing `Channel` and added checks T6 and T7 to `Main`. **What stayed unchanged:** `Notification`, `Reminder`, `UrgentAlert`, the `Channel` interface, `EmailChannel` and `SmsChannel`. The file extension.diff shows only `src/Main.java` and `src/channel/PushChannel.java` changed.

Trade-off. Bridge adds an extra interface and an extra level of indirection. For a small program with only one or two channels this is more structure than a simple class needs, and a reader must follow a call through two hierarchies to see what happens.

Bridge vs Adapter. The intent is different. *Bridge* is a design decision made up front: it separates an abstraction from its implementation so both can evolve independently (here, notification kinds and channels designed together). *Adapter* is applied after the fact: it makes an existing class with an incompatible interface work where another interface is expected. If `PushChannel` were a ready-made third-party push library with its own method names, I would write an `Adapter` that translates `deliver(...)` into that library's calls and plug the adapter into the existing `Bridge`.

5. Sources

[1] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (Bridge and Adapter).

[2] Refactoring.Guru, Bridge: <https://refactoring.guru/design-patterns/bridge>

[3] Refactoring.Guru, Adapter: <https://refactoring.guru/design-patterns/adapter>

[4] R. C. Martin, *Clean Code*, Prentice Hall, 2008.